Project Executable Files

Date	29 June 2026
Team ID	
Project Name	Rising Waters: Smart AI Flood Prediction System
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
Rising-Waters-Smart-AI-Flood-Prediction-System/
│
├── app.py                  # Main Flask Application
├── config.py               # Application Configuration
├── train_model.py          # Model Training Script
├── predict.py              # Prediction Logic
├── preprocess.py           # Data Preprocessing
├── requirements.txt        # Python Dependencies
├── README.md               # Project Documentation
├── Procfile                # Deployment Configuration
├── runtime.txt             # Python Runtime Version
│
├── model/
│   ├── flood_model.pkl     # Trained XGBoost Model
│   └── scaler.pkl          # Feature Scaler
│
├── dataset/
│   ├── flood_dataset.csv   # Training Dataset
│   └── cleaned_dataset.csv # Processed Dataset
│
├── templates/
│   ├── index.html
│   ├── predict.html
│   ├── result.html
│   └── about.html
│
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── script.js
│   └── images/
│
├── screenshots/
│   ├── home.png
│   ├── prediction.png
│   └── result.png
│
└── tests/
    └── test_model.py
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://predict-aqua-ai.lovable.app/
Login Credentials (Demo)	gjayaprakashreddy019@gmail.com
Platform / Hosting Provider	IBM Cloud / Render
Repository Link	
Demo Video Link	

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites

- Operating System: Windows 10/11, Linux, or macOS
- Python 3.10 or above
- pip (Python Package Manager)
- Visual Studio Code / PyCharm (Recommended)
- Flask Framework
- Required Python Libraries: scikit-learn, xgboost, pandas, numpy, matplotlib, joblib

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on the quality and completeness of historical weather data.	Use updated and validated datasets for better accuracy.
2	The system predicts flood risk based only on available meteorological features and does not consider live sensor or satellite data.	Future enhancement: Integrate IoT sensors and real-time weather APIs
3	Flask application supports moderate traffic and may require scaling for large-scale deployments.	Deploy on IBM Cloud with load balancing or containerization for production use.